

JÚLIA GABRIELA GOMES DA SILVA
LARA EWELLEN DE CARVALHO ROCHA
LUIZA LIMA DE SÁ
MARIA EDUARDA NUNES BEZERRA

**TRADUTOR DE LIBRAS PARA ÁUDIO COM INTELIGÊNCIA
ARTIFICIAL**

Relatório final atualizado, apresentado como
parte das exigências do projeto prático com
desenvolvimento de protótipo, referente ao
Tradutor de Libras para Áudio com
Inteligência Artificial.

BRASÍLIA
2026

RESUMO

Este trabalho apresenta a versão atualizada do protótipo funcional de tradução de Língua Brasileira de Sinais (Libras) para texto e áudio, utilizando visão computacional e aprendizado de máquina. O sistema captura imagens por webcam, extrai landmarks com MediaPipe e classifica sinais por meio de uma rede neural temporal composta por camadas Conv1D, LSTM bidirecional e mecanismo de atenção. A versão atual do modelo foi treinada com 13 classes de sinais isolados, incluindo Eu e Quero, e atingiu acurácia de 66,47% no conjunto de teste, com acurácia balanceada de 69,19%. A interface também forma frases a partir dos sinais detectados, permitindo consultar, limpar, desfazer o último sinal e reproduzir a frase completa por síntese de voz.

Palavras-chave: Libras. Visão computacional. Aprendizado de máquina. Acessibilidade. Síntese de voz.

SUMÁRIO

RESUMO.....	2
SUMÁRIO.....	3
1 OBJETIVO PRINCIPAL.....	4
2 DESCRIÇÃO DO PROBLEMA E RELEVÂNCIA.....	5
3 ESCOPO DO PROTÓTIPO.....	6
4 ORGANIZAÇÃO DO PROJETO.....	7
5 TECNOLOGIAS UTILIZADAS E JUSTIFICATIVA.....	8
6 BASE DE DADOS E VERIFICAÇÃO.....	9
7 PRÉ-PROCESSAMENTO E EXTRAÇÃO DE CARACTERÍSTICAS.....	10
7.1 NORMALIZAÇÃO DOS LANDMARKS.....	10
7.2 JANELAS DE VÍDEO.....	10
8 MODELO DE INTELIGÊNCIA ARTIFICIAL.....	11
8.1 ARQUITETURA DO MODELO.....	11
8.2 DATA AUGMENTATION E AJUSTES CONTRA CONFUSÃO.....	11
9 FUNCIONAMENTO DO PIPELINE.....	12
10 FORMAÇÃO DE FRASES E SÍNTESE DE VOZ.....	13
11 RESULTADOS E MÉTRICAS.....	14
12 DIFICULDADES ENCONTRADAS.....	16
13 ANÁLISE CRÍTICA.....	17
14 TRABALHOS FUTUROS.....	18
15 COMO EXECUTAR O PROJETO.....	19
16 CONCLUSÃO.....	20
17 REFERÊNCIAS.....	21
ANEXOS.....	22

1 OBJETIVO PRINCIPAL

O objetivo deste projeto é desenvolver um protótipo funcional de um sistema baseado em Inteligência Artificial capaz de reconhecer sinais de Libras a partir da câmera de um computador e converter o resultado em texto e áudio.

O protótipo reconhece sinais isolados em tempo real. O sistema captura imagens da webcam, extrai pontos de referência do corpo, das mãos e do rosto, organiza esses dados como uma sequência temporal, classifica o sinal e apresenta o resultado na interface web.

2 DESCRIÇÃO DO PROBLEMA E RELEVÂNCIA

A comunicação entre pessoas surdas que utilizam Libras e pessoas ouvintes que não conhecem a língua ainda depende, em muitos contextos, da presença de intérpretes ou de alternativas improvisadas de comunicação.

O uso de computadores e smartphones com câmera torna possível propor soluções de apoio mais acessíveis. A ideia central do projeto é usar a câmera como sensor, transformar o movimento do usuário em dados numéricos e aplicar um modelo de IA para reconhecer os sinais.

3 ESCOPO DO PROTÓTIPO

Quadro 1 – Funcionalidades implementadas no protótipo

Funcionalidade	Implementação no projeto
Captura de vídeo	Interface web acessa a webcam pelo navegador usando getUserMedia.
Rastreamento corporal	Backend usa MediaPipe para extrair landmarks de mãos, pose corporal e rosto.
Classificação dos sinais	Modelo TensorFlow/Keras classifica sequências de 20 frames.
Geração de texto	Interface mostra sinal reconhecido, confiança, histórico e top predições.
Formação de frases	Sinais aceitos são acumulados no backend e transformados em uma frase sugerida.
Síntese de voz	Frontend usa Web Speech API para falar sinais isolados e a frase completa.
Feedback humano	Interface registra acertos e correções em novas amostras rotuladas.

Fonte: elaborado pelas autoras (2026).

O vocabulário atual do modelo possui 13 classes.

Quadro 2 – Classes reconhecidas pelo modelo

Classe	Papel no sistema
Aceitar	Sinal reconhecido
Banheiro	Sinal reconhecido
Bebida	Sinal reconhecido
Calmo	Sinal reconhecido
Casa	Sinal reconhecido
Desconhecido	Classe de rejeição
Eu	Sinal reconhecido
Livro	Sinal reconhecido
Nome	Sinal reconhecido
Obrigado	Sinal reconhecido
Oi	Sinal reconhecido
Quero	Sinal reconhecido
Sortudo	Sinal reconhecido

Fonte: elaborado pelas autoras (2026).

4 ORGANIZAÇÃO DO PROJETO

Quadro 3 – Estrutura de arquivos e pastas do projeto

Arquivo/Pasta	Função
app.py	Backend Flask responsável por receber frames, extrair features, retornar predições e manter a frase detectada.
frase_builder.py / rag_libras.py	Módulos usados para formatar sinais e sugerir frases a partir da sequência detectada.
validar_dataset.py	Verifica padrão dos nomes, abertura dos vídeos, frames e quantidade por classe.
etapa2_preprocessamento.py	Extrai landmarks dos vídeos e gera data/landmarks.csv.
etapa3_treinamento.py	Treina o modelo, calcula métricas e salva artefatos.
front/	Interface web com câmera, histórico, frase, voz e feedback.
models/	Modelo treinado, encoder de labels e thresholds.
reports/	Métricas, matriz de confusão e gráficos de treinamento.

Fonte: elaborado pelas autoras (2026).

5 TECNOLOGIAS UTILIZADAS E JUSTIFICATIVA

Quadro 4 – Tecnologias utilizadas no desenvolvimento do protótipo

Tecnologia	Uso	Justificativa
Python 3.11	Backend, pré-processamento e treinamento	Ecossistema amplo para IA e visão computacional.
Flask	Servidor web	API simples para receber frames, retornar previsões e controlar a frase.
OpenCV	Manipulação de imagens	Decodifica frames e prepara imagens.
MediaPipe	Extração de landmarks	Detecta mãos, pose e rosto em tempo real.
TensorFlow/Keras	Modelo neural	Implementa Conv1D, LSTM e atenção temporal.
Scikit-learn	Métricas e labels	Gera acurácia, relatório e matriz de confusão.
Web Speech API	Síntese de voz	Fala os sinais reconhecidos e a frase completa no navegador.

Fonte: elaborado pelas autoras (2026).

A escolha do MediaPipe foi importante porque ele transforma imagens em landmarks numéricos. Isso reduz a complexidade do problema, pois o modelo aprende a partir de coordenadas de mãos, corpo e rosto em vez de pixels brutos.

6 BASE DE DADOS E VERIFICAÇÃO

Os vídeos ficam em videos/data/ e seguem o padrão NomeDoSinal_ArticuladorN.mp4.

Quadro 5 – Quantidade de vídeos por classe

Classe	Vídeos
Aceitar	20
Banheiro	20
Bebida	22
Calmo	20
Casa	20
Desconhecido	63
Eu	27
Livro	20
Nome	20
Obrigado	20
Oi	20
Quero	28
Sortudo	20

Fonte: elaborado pelas autoras (2026).

Antes do pré-processamento, o projeto executa uma etapa de verificação com `validar_dataset.py`. Essa etapa confere se a pasta de vídeos existe, se os arquivos usam extensões de vídeo aceitas, se o nome segue o padrão `Nome_ArticuladorN`, se o arquivo abre corretamente com OpenCV, se possui frames válidos, além de calcular quantidade de vídeos por sinal e frames mínimo, médio e máximo.

A verificação também emite avisos para sinais com menos de 10 vídeos e indica que o ideal é possuir pelo menos 20 vídeos por classe. Isso evita iniciar o pré-processamento final com arquivos quebrados, nomes fora do padrão ou classes muito desbalanceadas.

7 PRÉ-PROCESSAMENTO E EXTRAÇÃO DE CARACTERÍSTICAS

Cada frame é convertido em um vetor com 288 features, combinando pose corporal, mão esquerda, mão direita e pontos selecionados do rosto.

Quadro 6 – Composição do vetor de características por frame

Grupo de landmarks	Quantidade	Valores por ponto	Total
Pose corporal	33 pontos	x, y, z, visibilidade	132
Mão esquerda	21 pontos	x, y, z	63
Mão direita	21 pontos	x, y, z	63
Rosto selecionado	10 pontos	x, y, z	30
Total	-	-	288

Fonte: elaborado pelas autoras (2026).

O sistema trabalha com sequências de 20 frames, formando uma entrada de 20 x 288 para o modelo.

7.1 NORMALIZAÇÃO DOS LANDMARKS

Os landmarks são normalizados para reduzir variações de enquadramento, distância da câmera e tamanho da pessoa.

7.2 JANELAS DE VÍDEO

O pré-processamento divide os vídeos em janelas de 20 frames e avalia critérios como detecção de mãos, movimento e taxa de aceitação.

8 MODELO DE INTELIGÊNCIA ARTIFICIAL

O projeto usa aprendizado supervisionado: cada sequência de landmarks é associada a um rótulo, como Oi, Casa, Eu ou Quero.

8.1 ARQUITETURA DO MODELO

A arquitetura combina Conv1D, Batch Normalization, Dropout, duas camadas LSTM bidirecionais, mecanismo de atenção temporal, camada densa e saída softmax.

8.2 DATA AUGMENTATION E AJUSTES CONTRA CONFUSÃO

Quadro 7 – Distribuição de amostras por conjunto

Conjunto	Amostras
Treino original	1.398
Validação	280
Teste	498
Treino após augmentation	11.781

Fonte: elaborado pelas autoras (2026).

As augmentações incluem perturbações controladas nos landmarks, variações de escala, deslocamento, velocidade e dropout de pontos.

Para lidar com confusões recorrentes, o treinamento atual usa label smoothing de 0,01 e pesos/amostras focadas. A classe Oi recebeu multiplicador de amostras 2 e peso de classe 2,2; Eu recebeu multiplicador de amostras 2 e peso de classe 1,7. Mesmo assim, Oi continua sendo um dos sinais mais difíceis.

9 FUNCIONAMENTO DO PIPELINE

O pipeline segue: câmera ou vídeo, captura de frames, extração de landmarks com MediaPipe, normalização, sequência de 20 frames, classificação pelo modelo temporal, filtros de estabilidade, texto, formação de frase e síntese de voz.

Em tempo real, o navegador captura frames, envia para /predict, o Flask decodifica com OpenCV, o MediaPipe extrai landmarks e o modelo prediz probabilidades. O backend aplica thresholds, votação, margem, rejeição por desconhecido, regras de confusão sensível e debounce antes de aceitar um sinal.

10 FORMAÇÃO DE FRASES E SÍNTESE DE VOZ

A versão atual do sistema não apenas mostra sinais isolados: ela mantém uma lista de sinais aceitos no backend em `frase_detectada`. Quando um sinal final é aceito, ele é formatado e adicionado à lista da frase. Se o mesmo sinal aparece repetido consecutivamente, o backend ignora a repetição para evitar duplicidade.

Quadro 8 – Recursos de formação e controle da frase

Recurso	Funcionamento
Lista de sinais	Sinais aceitos são guardados no backend e enviados ao frontend como <code>sinais_detectados</code> .
Frase sugerida	A sequência de sinais é enviada para <code>gerar_frase_com_rag</code> , que retorna <code>frase_sugerida</code> , origem e explicação.
Consulta da frase	A rota <code>GET /phrase</code> retorna o estado atual da frase.
Limpar frase	A rota <code>POST /phrase/clear</code> apaga a sequência detectada.
Desfazer último sinal	A rota <code>POST /phrase/undo</code> remove o último sinal aceito.
Falar frase	O frontend usa <code>speechSynthesis</code> para reproduzir a frase completa em pt-BR.

Fonte: elaborado pelas autoras (2026).

No frontend, a frase aparece com a lista de sinais, uma frase sugerida, a origem da sugestão e uma explicação. Também existem controles para falar a frase completa, interromper a voz, alternar voz automática, limpar frase e remover o último sinal.

11 RESULTADOS E MÉTRICAS

Quadro 10 – Métricas gerais do modelo no conjunto de teste

Métrica	Resultado
Acurácia final	66,47%
Acurácia balanceada	69,19%
Amostras de teste	498
Classes avaliadas	13

Fonte: elaborado pelas autoras (2026).

Quadro 11 – Métricas de classificação por classe

Classe	Precisão	Recall	F1-score	Suporte
Aceitar	0,35	0,62	0,45	24
Banheiro	0,78	0,85	0,82	62
Bebida	0,78	0,75	0,77	53
Calmo	0,74	0,67	0,70	42
Casa	0,69	0,78	0,73	23
Desconhecido	0,48	0,52	0,50	44
Eu	0,67	1,00	0,80	14
Livro	0,69	0,54	0,61	46
Nome	0,76	0,85	0,80	26
Obrigado	0,81	0,63	0,71	60
Oi	0,50	0,26	0,34	47
Quero	0,58	0,72	0,65	29
Sortudo	0,71	0,79	0,75	28

Fonte: elaborado pelas autoras (2026).

A acurácia de 66,47% e a acurácia balanceada de 69,19% mostram desempenho compatível com um protótipo funcional com 13 classes e formação de frases integrada à aplicação.

As classes com melhor desempenho foram Banheiro, Bebida, Casa, Eu, Nome, Obrigado e Sortudo. A principal dificuldade continua sendo Oi, que teve recall de 0,26 e F1-score de 0,34.

Quadro 12 – Principais confusões entre classes

Quantidade	Classe real	Classe prevista
15	Oi	Desconhecido

Quantidade	Classe real	Classe prevista
11	Livro	Aceitar
9	Oi	Banheiro
8	Calmo	Aceitar
6	Obrigado	Desconhecido
6	Desconhecido	Eu
5	Obrigado	Oi

Fonte: elaborado pelas autoras (2026).

12 DIFICULDADES ENCONTRADAS

Quadro 13 – Principais dificuldades e impactos no projeto

Dificuldade	Impacto
Dataset limitado	Poucos exemplos por classe dificultam a generalização.
Variação entre articuladores	Pessoas diferentes executam o mesmo sinal com diferenças.
Iluminação e enquadramento	Podem afetar a detecção dos landmarks.
Sinais parecidos	Alguns gestos compartilham trajetórias ou posições semelhantes.
Reconhecimento em tempo real	Predições quadro a quadro podem oscilar.
Classe Desconhecido	É difícil representar todos os movimentos fora do vocabulário.
Sinal Oi	Ainda é confundido principalmente com Desconhecido, Banheiro e Casa, conforme a matriz de confusão atual.

Fonte: elaborado pelas autoras (2026).

O caso de Oi merece destaque porque, mesmo após aumento de peso no treinamento e regras de correção prioritária, ele ainda aparece como o principal ponto fraco. No relatório de confusões atual, Oi foi classificado como Desconhecido 15 vezes, como Banheiro 9 vezes e como Casa 4 vezes. Isso sugere que o modelo ainda não separa bem a trajetória e a configuração espacial desse sinal em relação a gestos visualmente próximos.

13 ANÁLISE CRÍTICA

O protótipo atende ao objetivo de demonstrar um pipeline completo de IA para tradução de sinais isolados de Libras para texto, frase sugerida e áudio.

A acurácia de 66,47% é adequada para uma demonstração acadêmica, mas ainda não permite afirmar que o sistema esteja pronto para uso real de acessibilidade sem supervisão. A formação de frases amplia a utilidade do protótipo, mas também depende diretamente da qualidade dos sinais detectados.

Libras envolve estrutura gramatical própria, expressões faciais, contexto, direção do olhar, configuração das mãos, orientação, ponto de articulação e movimento. O protótipo ainda trabalha com sinais isolados, e a frase sugerida é uma aproximação textual gerada a partir desses sinais.

14 TRABALHOS FUTUROS

Quadro 14 – Melhorias propostas e benefícios esperados

Melhoria	Benefício esperado
Aumentar o dataset	Melhorar acurácia e generalização.
Coletar mais exemplos de Oi	Reduzir confusão com Desconhecido, Banheiro e Casa.
Gravar mais pessoas	Reduzir dependência de um estilo específico.
Variar ambiente e iluminação	Tornar o modelo mais robusto.
Melhorar a classe Desconhecido	Reduzir falsas traduções.
Expandir vocabulário	Reconhecer mais sinais.
Melhorar a geração de frases	Produzir frases mais naturais e coerentes.
Segmentar início e fim	Melhorar reconhecimento em fluxo contínuo.
Testar Transformer ou TCN	Comparar arquiteturas modernas.
Exportar para TensorFlow Lite	Facilitar execução em dispositivos móveis.

Fonte: elaborado pelas autoras (2026).

15 COMO EXECUTAR O PROJETO

Preparar Python 3.11, criar e ativar a .venv, instalar requirements.txt e executar: `validar_dataset.py`, `etapa2_preprocessamento.py`, `etapa3_treinamento.py` e `app.py`. Depois, acessar <http://localhost:5000>.

16 CONCLUSÃO

O projeto desenvolveu um protótipo funcional de Tradutor de Libras com IA. A versão atual trabalha com 13 classes, forma frases a partir dos sinais aceitos e atingiu 66,47% de acurácia no conjunto de teste e 69,19% de acurácia balanceada.

A abordagem é promissora e mostra evolução em relação à integração do sistema, especialmente pela validação do dataset, feedback humano, controle de frase e síntese de voz. Ainda assim, o sinal Oi permanece como dificuldade relevante e deve ser priorizado em futuras coletas e ajustes do modelo.

17 REFERÊNCIAS

IA LIBRAS. Página inicial. Disponível em: <https://ialibras.com.br/>. Acesso em: 16 jun. 2026.

HAND TALK. Página inicial. Disponível em: <https://www.handtalk.me/br/>. Acesso em: 16 jun. 2026.

MEDIAPIPE. Documentação oficial. Disponível em: <https://developers.google.com/mediapipe>. Acesso em: 16 jun. 2026.

TENSORFLOW. Documentação oficial. Disponível em: <https://www.tensorflow.org/>. Acesso em: 16 jun. 2026.

KERAS. Documentação oficial. Disponível em: <https://keras.io/>. Acesso em: 16 jun. 2026.

SCIKIT-LEARN. Documentação oficial. Disponível em: <https://scikit-learn.org/>. Acesso em: 16 jun. 2026.

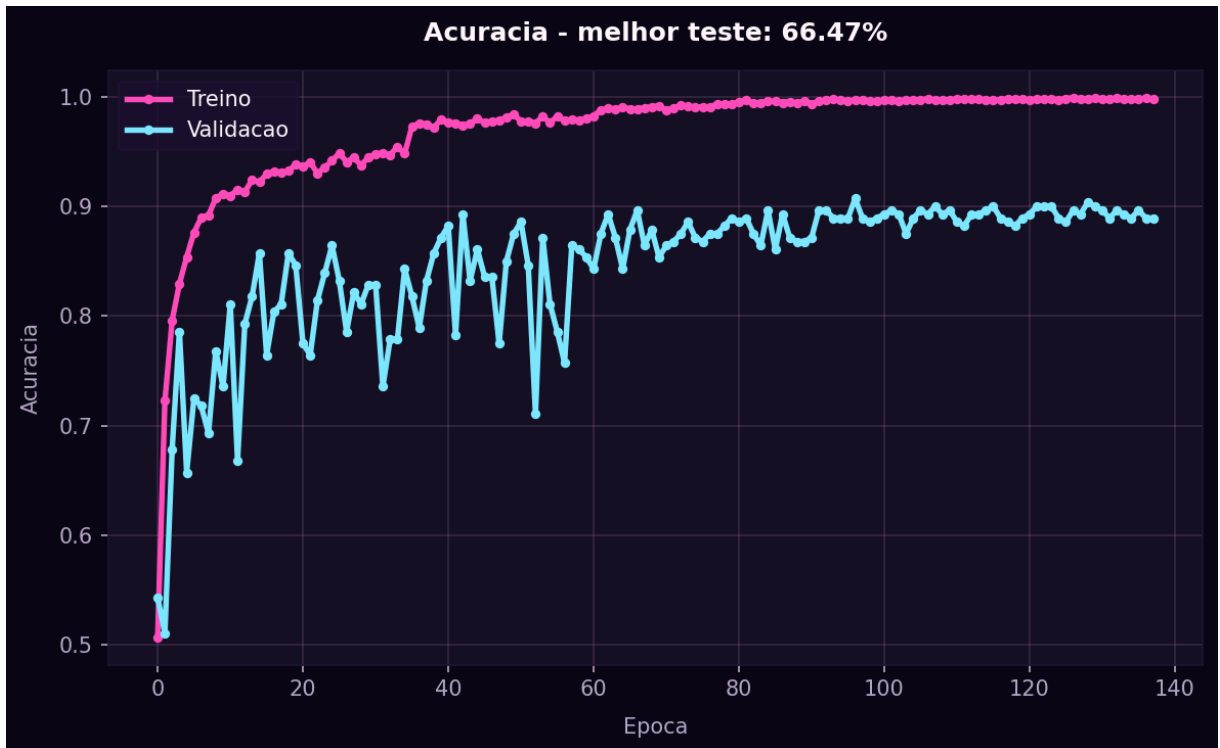
FLASK. Documentação oficial. Disponível em: <https://flask.palletsprojects.com/>. Acesso em: 16 jun. 2026.

MDN WEB DOCS. Web Speech API. Disponível em: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API. Acesso em: 16 jun. 2026.

UNIVERSIDADE FEDERAL DE PERNAMBUCO. Base de vídeos de Libras. Disponível em: <https://libras.cin.ufpe.br/>. Acesso em: 16 jun. 2026.

ANEXOS

ANEXO A – GRÁFICO DE ACURÁCIA DO TREINAMENTO



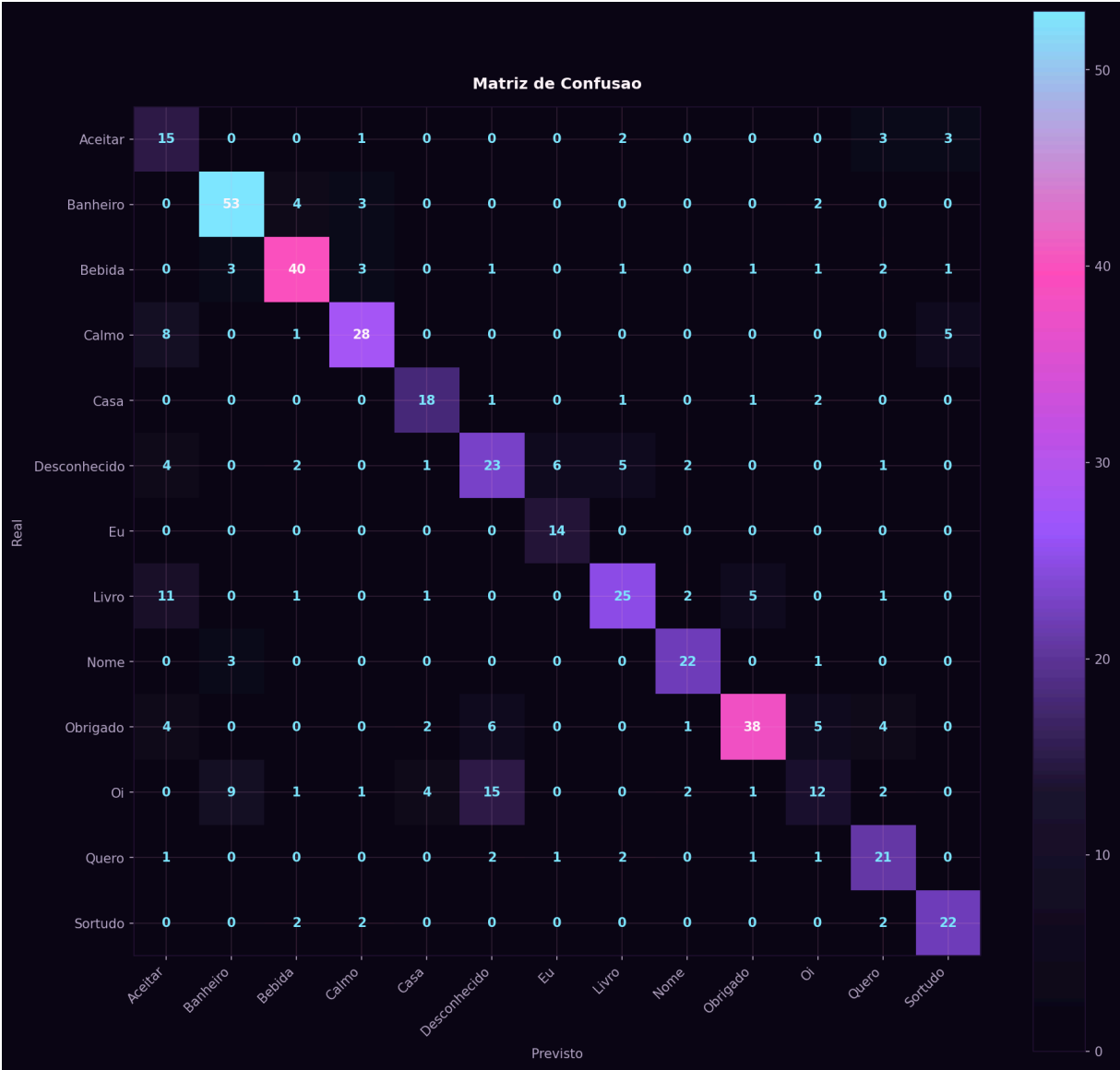
Fonte: elaborado pelas autoras (2026), a partir dos registros de treinamento do modelo.

ANEXO B – GRÁFICO DE PERDA DO TREINAMENTO



Fonte: elaborado pelas autoras (2026), a partir dos registros de treinamento do modelo.

ANEXO C – MATRIZ DE CONFUSÃO



Fonte: elaborado pelas autoras (2026), a partir dos resultados do conjunto de teste.